reason why we have to create our own synthetic data for training. We start with the training data for IFEval created in Tulu (Lambert et al., 2025), consisting of 15k examples formatted for IFEval with its verifiable rules. However, these datapoints are designed for reinforcement learning with verifiable rewards (RLVR), and therefore do not provide a reference on how to solve the instructions. We use a model trained via RLVR on this dataset, Olmo3 32B Thinking, to generate 10 candidate solutions for every datapoint in Tulu; we then filter these to keep only the correct ones, and remove duplicate answers. Our final training set contains 37k datapoints.

D. Hyperparameters

In this section, we provide a short description of the parameters we perform a search over in our experiments, when trying to find Pareto optimal length–performance pairs (b, δ). The full results of our hyperparameter sweeep is in Figure 9. The hyperparameters used are:

For ICL. We sweep over the number of examples n used in the in-context learning experiments. We increase this number until we run into memory limits on a single 80GB GPU.

For URIAL. We perform no sweep for this method. We run our experiments with a hand-designed prompt to explain the task, and we run this method using $n = 2$ examples of the data.

For Subset Training. We again sweep this method over the number of examples n used. We experiment with subsets of the data with sizes in powers of 2, starting with $n = 8$ and finishing with $n = 32768$ examples. We additionally experiment with fine-tuning the model using the full dataset. For each experiment, we also use learning rates 10^{-4} and 10^{-5}.

For LoRA. We use learning rates 10^{-4} and 10^{-5}, always training with the full dataset.

For B-LoRA. We use scale learning rates of 10^{-2} and 2×10^{-3}, training on the full dataset. We do one experiment with rank 1 for every LoRA layer and no rank pruning, and another starting with rank 2, but pruning the rank according to the optimisation objective of the method (Meo et al., 2024).

E. The Data used in Pre- and Post-training

Here, we detail the data used to pre- and post-train our analysed models.

SmolLM3. Bakouch et al. (2025) report using 8 trillion tokens during the `stage-1` of training (i.e., for pre-training). Assuming each token to be 4 bytes (enough to encode each vocabulary item of 128k in its tokenizer), this implies 32 terabytes of data for pre-training. Bakouch et al. also report using 3.2 trillion tokens during `stage-2`, `stage-3`, and the long context training stage (which we will here refer, jointly, to mid-training). This amounts to a total of 12.8 terabytes of data. Finally, Bakouch et al. report using a total of 37.5 billion unique training tokens for post-training, which amounts to 150 gigabytes of data, which was gathered by combining multiple post-training datasets. In this data (which they name the SmolTalk2 dataset), they include data of verifiable instruction following based on rules of IFEval, and also synthetic questions sourced from GSM8K training set.

Olmo3 7B. Olmo et al. (2025) report using 5.93 trillion tokens in the `stage-1` of training (i.e., for pre-training). This amounts to 23.7 terabytes of data for pre-training. Olmo et al. also report using a total of 150 billion tokens for `stage-2` and long context specialized training (i.e., for mid-training). This amounts to 600 gigabytes of data. Finally, Olmo et al. report using a total of 45 billion tokens in the post-training of the Olmo3 family. This amounts to 180 gigabytes of data. This includes stages of supervised fine-tuning, and also reinforcement learning with verifiable rewards. They release different variants of post-trained checkpoints, and we opt to use the last checkpoint in the `instruct` version.

F. Extra Results

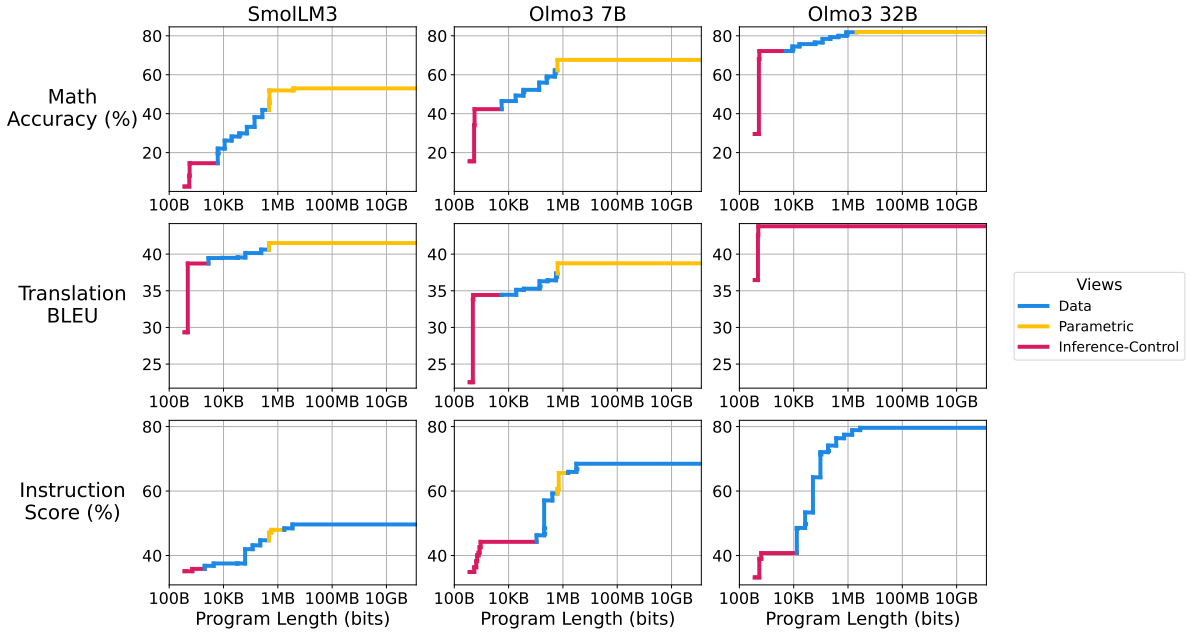

Figure 6. Length–performance Pareto curve of SmolLM3 3B, Olmo3 7B, and Olmo3 32B in Math (GSM8K), Translation (FLORES) and Instruction (IFEval). Different views of superficiality inform different regions of the Pareto curve, with some variance depending on the choice of model and task.

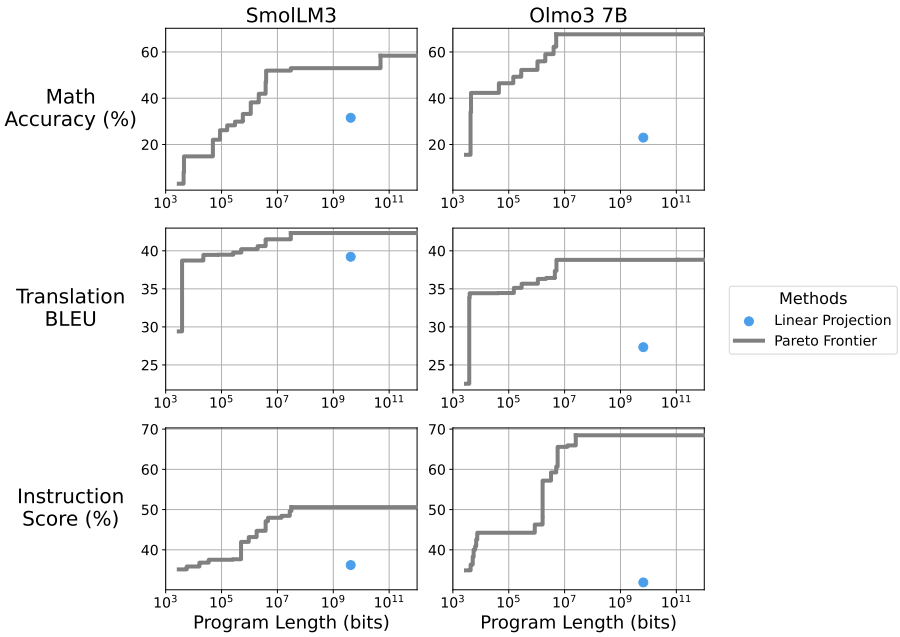

Figure 7. Performance of the linear (head-only) fine-tuning approach proposed by Chen et al. (2025) and our estimated Pareto curves in Math (GSM8K), Translation (FLORES) and Instruction (IFEval).

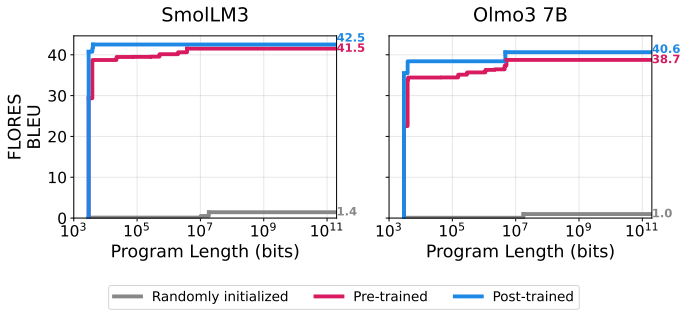

Figure 8. Estimated complexity curves conditioned on pre- and post-trained checkpoints of SmolLM3 and Olmo3 for FLORES.

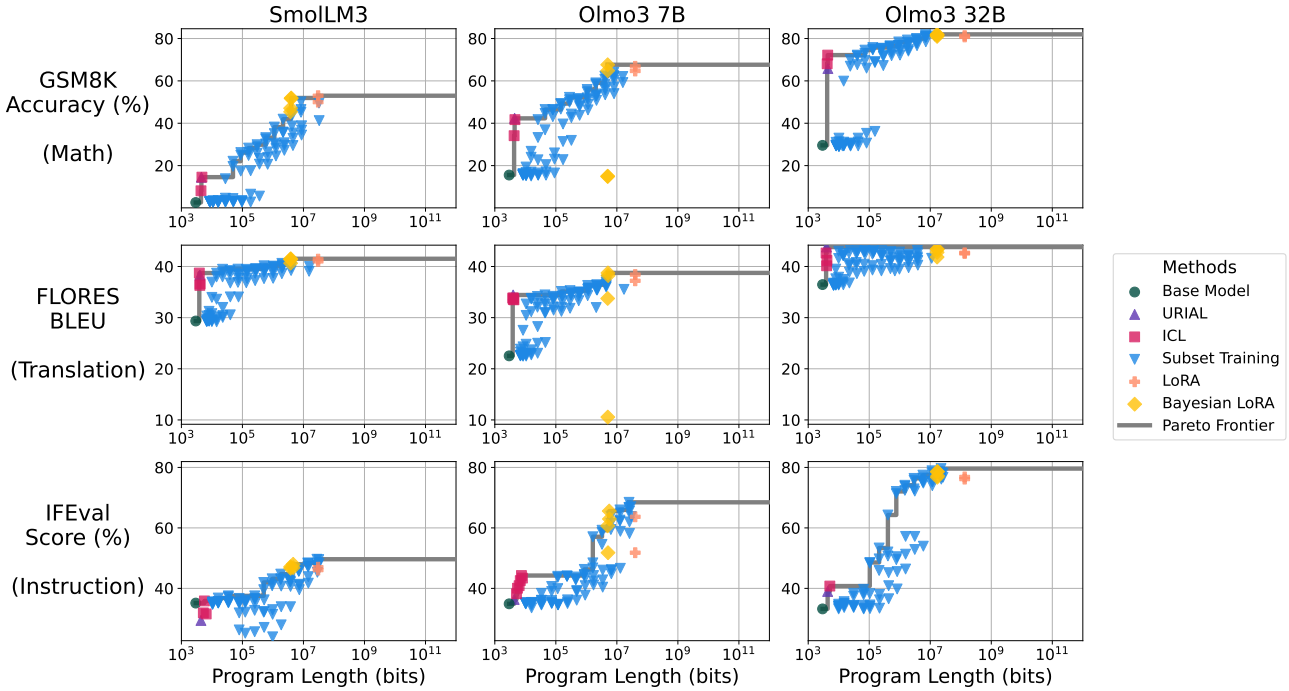

Figure 9. The program lengths vs. performance Pareto curves for SmolLM3 and Olmo3 in GSM8K and FLORES. We test each method described in Section 4, and show their message lengths and performances obtained. This is an extended version of Figure 3 with all the results from our hyperparameter sweep.

G. Combining the Parametric and the Data View

In section Section 4, we showcased how every argument for superficiality can be unified under our task complexity lens. Each argument (the data view, the parametric view, and the inference-control view) proposes different strategies with which to build a program P with small length to adapt a model for a task. Since these strategies are roughly orthogonal, we hypothesise that we can compose them together to achieve tighter Pareto curves of (b, δ) values. Here, we develop a method to combine the ② parametric view with the ① data view, in which the program P needs to encode only few batches of the task, and few parameters which will be used to decide how to weight the gradient that each example produces.

Let θ be the pre-trained model, and $\mathcal{D}$ be the full training dataset of task T. We draw a small subset $\mathcal{D}' \subset \mathcal{D}$ and we fine-tune θ on $\mathcal{D}'$, collecting each weight update $\Delta\theta_i$ during this process; here, i indexes each batch observed in this fine-tuning run (from 1 to the number of batches in $\mathcal{D}'$). Now, let k represent a linear projection layer in the analysed model, and θ^k and b^k represent the weights and bias of this layer. We initialise parameters $\alpha_i^k = 1$ which will be used to balance the updates of different $\Delta\theta_i$. We modify the forward pass of every layer to make use of these new α_i^k parameters. Originally, the forward